

AI-powered semantic product search and recommendation system built using Retrieval-Augmented Generation (RAG) and Vector Databases

By : Surya Jade Edakunni

Independent Study - INSY 661

Instructor: Professor Animesh Animesh

Date: 28th August 2026

I.) Introduction & Background

"I'm looking for a moisturizer for dry, sensitive skin under $40 that won't clog my pores."

In a physical retail store, a customer can ask a sales associate a question like this and receive personalized recommendations based on their needs. However, in online shopping, customers often rely on keyword-based search engines that may fail to capture the intent behind natural language queries. With the growth of e-commerce, retailers are increasingly adopting AI-powered shopping assistants capable of understanding conversational requests and providing personalized product recommendations. The beauty industry specifically is projected to grow annually by 5% to $590 billion USD by 2030 (McKinsey & Company, 2026).

The beauty industry offers a large range of products, making it increasingly difficult for consumers to identify those that best meet their needs. This brings forth a challenge for one of the beauty industry's most important consumer demographics: Gen Z. According to Statista Research Department (2025), although they are the newest generation to enter the workforce, they spend just over $9,000 USD annually on beauty products, trailing Millennials by only around $400. Additionally, nearly half of Gen Z express that they find it more difficult to make decisions today than last year, with 48% regretting their purchases (Retail Insider, 2026).

As a highly connected generation, Gen Z increasingly discovers and purchases beauty products through online platforms. The quality of digital shopping experiences is a critical factor in influencing purchase decisions. While e-commerce provides consumers with greater accessibility and product variety, it also introduces new challenges associated with navigating complex product catalogues. Unlike physical retail environments where customers can directly interact with sales associates, online shoppers must often rely on search functions and recommendation systems to identify suitable products. According to McKinsey & Company (2026), e-commerce is expected to become the most used beauty purchasing channel, although successful retailers will continue adopting an omnichannel approach. As competition

increases, brands that fail to provide differentiated shopping experiences risk losing customers to competitors offering greater convenience and value.

The implementation of AI-powered shopping assistants presents an opportunity for retailers to enhance the online shopping experience by improving personalization and customer decision-making. By understanding consumer needs and providing recommendations tailored to individual preferences, these systems can help reduce the friction associated with online shopping. McKinsey & Company (2026) estimates that AI platforms could influence up to 35% of e-commerce transactions over the next three to five years, highlighting the potential impact of these technologies on digital retail. Similarly, Amazon reported that more than 250 million customers used its AI shopping assistant Rufus between 2024 and 2025, with customers who interacted with the assistant being over 60% more likely to make a purchase during that shopping journey (Amazon, 2025). Furthermore, Salesforce found that 46% of consumers trust recommendations provided by AI assistants (Parton, 2025). The role of online shopping is moving beyond helping consumers locate products toward creating experiences that better support individual customer needs.

Sam Galanis, sales director at Pinterest (Retail Insider, 2026), says “This is no longer just about answering an explicit search query; it is about showing up early, anticipating intent, supporting exploration and helping consumers find products that reflect who they are and how they want to express themselves.” Meeting these evolving expectations requires retailers to move past traditional search systems. This shift has also transformed how product information is stored and retrieved.

II.) Evolution of Database Technologies

Progressing toward semantic search is part of a much longer development in the way data is stored and retrieved. Earlier database systems were designed for structured fields and clearly defined relationships. Over time, the way people interact with digital systems changed. Consumers began searching in less predictable ways, creating a need for technologies that could handle more flexible data and, eventually, understand the meaning behind a search.

The earliest database systems, including hierarchical and network databases, emerged during the

1960s and 1970s. These systems organized data around predefined relationships. Hierarchical databases followed a parent-child structure, while network databases allowed records to have more complex relationships with one another. These systems worked well for the business applications of the time, but changing the underlying data structure could be difficult. The relationships between different pieces of information generally had to be defined in advance. This became more limiting as organizations began working with content that was more varied and changed more frequently.

Relational databases addressed many of these problems. Information could be organized into tables and connected through relationships between fields. SQL provided a standardized way to retrieve and manipulate it. Their consistency and ability to work with structured data made relational databases particularly useful for business applications. Retailers, for example, could use them to store product prices, inventory, customer information, and transaction records. Queries could then be used to find products within a certain price range, identify items in a particular category, or retrieve a customer's previous purchases. For this type of structured content, relational databases remain highly effective.

The limitation becomes more apparent when a search depends on meaning rather than an exact value. Relational databases are built around relationships between fields and query conditions rather than an inherent understanding of the meaning of language. A search for "moisturizer for dry skin," for example, may not retrieve a product described as a "hydrating facial cream for dehydrated skin" if the system is relying primarily on matching exact phrases or logical conditions. A person can understand that these products may satisfy the same need, but a traditional database does not automatically make that connection.

The growth of web applications created another need. Companies were dealing with much larger volumes of data, and the structure of that data was becoming less predictable. NoSQL databases such as MongoDB and Cassandra became popular because they offered greater flexibility in how records could be stored and could support horizontal scaling across distributed systems. This made NoSQL databases useful for applications that needed to handle large amounts of content without relying on a fixed relational structure. However, NoSQL databases mainly addressed flexibility and scale. They did not solve the

problem of understanding what a user meant when entering a search query. A database can be flexible in how it stores a product while still relying on literal terms when retrieving it.

Machine learning introduced new ways of representing language that changed how semantic search could be performed. Beginning with approaches such as Word2Vec and continuing with transformer models such as BERT and Sentence-BERT, researchers developed ways to represent language as numerical vectors. These representations capture relationships between words and pieces of text based on how they are used. Phrases with different wording still have similar vector representations when they express similar meanings. For product search, this means that a query can be compared with product descriptions based on the concepts they represent rather than only the specific terms they contain.

Vector databases developed to make this type of retrieval practical. Systems such as Pinecone and ChromaDB are designed to store and search high dimensional vector representations efficiently. A product description can first be converted into an embedding, which is a numerical representation of its meaning. When a consumer enters a search query, the query can be converted into an embedding using the same general approach. The system can then compare the query vector with the product vectors and retrieve products that are semantically similar. This allows a search system to identify relevant products even when the wording of the query and the wording of the product description varies.

Vector databases should not be viewed as replacements for relational databases. Each technology serves a distinct purpose. A retailer may still rely on a relational database to store a product's price, inventory, rating, and product ID, while using a vector database to retrieve products based on the meaning of a customer's search. The two approaches can therefore work together. Vector databases add an alternative retrieval capability rather than replacing the role of a relational database. In this context, vector-based product search can focus on semantic relevance rather than simply matching the words contained in a query and product description.

III.) Technical Concepts

Vector Databases

Vector databases are specialized database systems that store and retrieve information using vector embeddings. They do this by comparing the numerical representations of pieces of data and identifying those that are most similar. For example, a product description such as “waterproof eyeliner for sensitive eyes” can be transformed into a vector containing hundreds or thousands of numerical values. Individual values within the vector do not have a direct interpretation. Taken together, however, they represent characteristics of the original text.

These numerical representations are generated through embedding models, which convert structured or unstructured data into vectors that capture contextual meaning. Because embedding models are trained differently, each model produces its own vector space, with differences in dimensions and how relationships between concepts are represented. This means that the same product description may produce different embeddings depending on the model used, including differences in vector dimensions and the way semantic relationships are captured. Therefore, the same embedding model must be used when creating both stored product embeddings and user query embeddings.

Vectorization and Embeddings

Before a product description can be compared to a customer's question, it must first be converted into a format a computer can process mathematically. This conversion process is called vectorization, and it is carried out by an embedding model rather than by the vector database itself.

The model learns these relationships during training by processing large amounts of text to place descriptions with similar meaning close to one another in this numerical space, while placing unrelated descriptions farther apart. This is why a query such as "what's good for dry skin" can retrieve a moisturizer even if the word "dry" never appears in the product's description. The model has learned that concepts like dryness, flaking, and hydration tend to occur in similar contexts, so it maps them to nearby regions of the vector space, and the two vectors end up sitting close together even without any shared vocabulary.

The embedding is only one part of what a vector database stores. Alongside the numerical vector,

the database keeps the original source text and any relevant metadata, such as a product name, category, or price. The vector is what makes similarity-based retrieval possible, but the original text and metadata are what allow the retrieved result to actually be interpreted, displayed, or filtered afterward. This difference is important when using a tool such as ChromaDB. In this implementation, ChromaDB provides the storage, indexing, and retrieval layer, while an external Sentence Transformer model creates the embeddings before they are added to the database.

Semantic Similarity and Distance Metrics

Once every product description and every user query exist as vectors, the system needs a mathematical way to decide which stored vectors are closest to the query vector.

Euclidean distance measures the straight-line distance between two vectors. Smaller values indicate greater similarity under this metric. To calculate it, the two vectors are lined up on top of one another, and the values are subtracted position by position. Each of these differences is then squared, all the squared differences are summed together, and finally the square root of that sum is taken. The result describes how far apart the endpoints of the two vectors are.

Cosine similarity works differently because it ignores the length of the vectors entirely and instead measures the angle formed between them. The calculation again begins by lining the two vectors up, but this time the corresponding values are multiplied together rather than subtracted, and those products are summed to produce a value that can be called x. Separately, each vector's own values are squared and summed, and the square root of each of those two sums is taken and multiplied together to produce a second value, y. Dividing x by y gives the cosine similarity. Because this calculation depends only on direction and not on magnitude, two vectors pointing the same way will be judged highly similar even if one is much longer than the other.

Inner product considers both direction and magnitude. It tends to favour vectors with larger magnitudes, meaning that a longer vector can appear more similar to the query simply because of its size rather than because it points in a more relevant direction. Once vectors have been normalized so that they all share the same length, however, inner product becomes mathematically identical to cosine similarity.

For this project, cosine distance was selected because the search compares text embeddings produced by a Sentence Transformer model. ChromaDB's default distance metric is the squared L2 norm, with cosine similarity and inner product available as alternative configuration options, so specifying cosine explicitly when the collection is created was a deliberate step rather than a default setting left unchanged.

Vector Indexing and Approximate Nearest Neighbour (ANN) Search

Comparing a query vector against every single embedding stored in the database, one at a time, would technically work, but this brute-force approach becomes computationally expensive as the number of stored vectors grows. To avoid this, vector databases rely on indexing structures that make it possible to find approximate nearest neighbours quickly, without checking every stored vector individually.

One of the most widely used indexing methods is Hierarchical Navigable Small World, commonly abbreviated as HNSW, which organizes stored vectors into a graph rather than a flat list. During a search, the algorithm explores promising neighbouring vectors rather than comparing the query with every stored vector. This reduces the number of distance calculations required to identify relevant results.

HNSW is designed to achieve high retrieval recall while reducing the number of distance calculations required during search. Recall measures the proportion of relevant nearest neighbours that are successfully retrieved. This efficiency comes with trade-offs, however, as the graph requires additional memory to store connections between vectors and can make updates more costly. Because each node maintains multiple edges connecting it to other nodes, the structure can take up considerably more storage space than simpler formats, particularly on storage media where latency is already a concern.

Rather than guaranteeing the mathematically exact closest matches every time, ANN search accepts a small amount of inaccuracy in exchange for a large gain in speed, which in turn translates into lower computational cost. For a smart search box intended to respond to a user's question in something close to real time, this trade-off is generally worthwhile, since a slightly imperfect but fast response is far more usable than a perfectly accurate response that takes too long to return.

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is an architecture that combines a retrieval system with a generative language model so that the model's output is grounded in a specific, external set of documents rather than only in what it learned during training. A standard LLM answers only from its internal parameters, which means it has no direct connection to any particular company's product catalogue and can produce answers that sound confident but have no real source behind them. RAG, however, addresses this by inserting a retrieval step before generation: a user's query is first used to search a database (in this case, a vector database) for the most relevant pieces of data, and only those retrieved pieces are handed to the language model as context. The model can then use the retrieved information as context when generating its response, reducing its reliance on data stored in its pretrained parameters. In this project, the Sephora product catalogue is embedded and stored in ChromaDB for retrieval, while a locally hosted Llama 3.1 model accessed through Ollama is used for query interpretation and final response generation.

Ollama

Ollama allows large language models to run locally rather than requiring requests to a hosted API. The model is loaded onto the local machine and inference is performed using the available CPU or GPU resources. In this project, Ollama is accessed through its Python library, which provides functions for interacting with the local model server.

A single LLM call is guided by a system prompt, an instruction given to the model before the user's actual input that shapes how it should behave. In many applications, it is tempting to give one large system prompt that asks the model to do several things at once, for example, understand what the user wants, retrieve relevant context, and then write a polished response. For this project, separating these responsibilities made it easier to control and test each stage independently. A common and more reliable design pattern is to split the work into multiple smaller calls, each with a short, narrowly scoped system prompt focused on exactly one task. One call might be responsible only for interpreting or rewriting the user's input into a cleaner form; a separate, later call is responsible only for turning retrieved information

into a final, readable answer.

IV.) Practical Implementation

Data Preparation and Feature Engineering

The dataset used for this implementation is the Sephora product dataset, obtained through kagglehub. Only *product_info.csv* was loaded, since the project is about product retrieval rather than review analysis. The raw file contained 8,494 products across 27 features. The dataset was first inspected for data types, missing values, distributions, and duplicates before preprocessing decisions were made. No duplicate product_id values existed, and no rows were fully duplicated. 79 product names appeared more than once, but these were legitimate product variations rather than errors, so they were kept. Columns with high missingness or limited value for retrieval were dropped: *sale_price_usd*, *value_price_usd*, *variation_desc*, *child_max_price*, *child_min_pric*e. Missing values in reviews and loves_count were replaced with 0, on the assumption that a missing value here means no recorded reviews or loves. Other missing fields were left as missing rather than filled in, since inventing values would introduce false knowledge into the dataset.

The remaining features were split into semantic data and metadata, with *product_id* kept in both to act as a linking key, similar to a primary key in a relational database. *brand_id* was kept as metadata rather than dropped, since it is a standardized identifier even though it carries no semantic meaning on its own; *brand_name* was used for semantics instead. The three category level columns were combined into a single category_path field using ">" as a separator, preserving the hierarchy in one string. A *product_text* field was then built by combining product name, brand, category, highlights, ingredients, and variation information. This became the input to the embedding model.

Embedding Generation and Storage in ChromaDB

Each *product_text* field was passed through the Sentence Transformers model *all-MiniLM-L6-v2*, producing one 384-dimensional embedding per product. For each product, the ChromaDB collection was populated with its ID, embedding, product text, and selected metadata (*price_usd*, *rating*, and *reviews*). Cosine distance was specified as the comparison metric, where a lower distance indicates greater semantic

similarity.

The persistent ChromaDB collection is stored locally within the *chroma_db* directory. Although this directory contains a *chroma.sqlite3* file, the application does not interact with the database through SQL. ChromaDB uses SQLite internally as part of its persistence architecture, while the application interacts with ChromaDB through its Python API for storing and retrieving product embeddings, documents, and metadata. The presence of SQLite therefore reflects part of ChromaDB's underlying storage implementation rather than a separate relational database used by the application.

One limitation I encountered with ChromaDB was its handling of missing metadata. Unlike relational databases, where NULL can be stored explicitly and queried using conditions such as IS NULL, ChromaDB metadata must use supported concrete value types. Missing numerical values therefore needed to be handled during preprocessing before the relevant metadata was stored in ChromaDB. Missing review counts, for example, were replaced with 0. This did not require re-embedding the products because the embeddings and metadata are stored separately. Storing metadata in ChromaDB does not mean that the metadata is automatically used during retrieval. Adding *price_usd*, *rating*, and *reviews* to ChromaDB does not cause the system to interpret terms such as "cheap" automatically. Numerical constraints must be explicitly provided through a metadata filter, such as *{"price_usd": {"$lte": 30}}*. A 0 in the metadata should also be read carefully, since it can mean either a genuine zero or a missing value, and the current system cannot tell the two apart. Query results were retrieved using *include = ["documents", "distances", "metadatas"]* so that all three pieces of information were available for the next stage.

```
print(product_embeddings.shape)
print(metadata_df.shape)
print(semantic_df.shape)
## Putting embeddings in Chromadb
client = ch.PersistentClient(path = "./chroma_db")

client.delete_collection(name = "SephoraProducts")

products = client.create_collection(
    name = "SephoraProducts",
    metadata = {"hnsw:space": "cosine"}
)

print("ChromaDB collection created successfully!")
```

```python
#Adding products to the vector Database

batch_size = 5000 #Largest bath size is 5000

for start in range(0, len(product_ids), batch_size):
    end = min(start + batch_size, len(product_ids))

    products.add(
        ids = product_ids[start:end],
        embeddings = product_embeddings[start:end].tolist(),
        documents = product_texts[start:end],
        metadatas = product_metadata[start:end])

    print(f"Added {start} to {end - 1}")
print("Products in ChromaDB:", products.count())
```

Figure 1. ChromaDB collection configuration and product storage

Teaching the System to Understand What the User Means

The first Ollama stage takes the user's natural-language request and turns it into a cleaner semantic query for the embedding model. For example, a request such as *"luxurious and feminine for summer"* can be reduced to *"luxurious, feminine, for summer"* before being passed to Sentence Transformers and mapped into the same vector space as the stored product embeddings.

Early attempts asked Ollama to extract more than just the semantic query. It was also asked to pull out *max_price*, *min_rating*, and *min_reviews* directly from the sentence. This did not work reliably. The model started inventing numbers that were never stated by the user. A phrase like *"highly rated"* was sometimes turned into an invented rating of 4.5, and vague popularity language occasionally produced invented review counts such as 5 or 10. Dollar amounts that were actually part of the descriptive request, such as $40 or $50, were also sometimes left inside the semantic query instead of being pulled out as a filter.

The final decision was therefore to keep the first Ollama stage focused on semantic interpretation rather than allowing it to invent deterministic numerical rules. Explicit numerical constraints can still be handled through ChromaDB metadata filters, but they need to be identified and passed to the retrieval function deliberately. The prototype therefore demonstrates the filtering capability separately from the semantic query interpretation rather than relying on the LLM to infer numerical thresholds from subjective language.

The prompt was also revised based on these failures. It instructed Ollama to use only context

stated by the user and prohibited unsupported inferences. For example, testing showed that the model could interpret “summer” as implying “beach,” so examples of these types of assumptions were added to the prompt. This made the model's behaviour more consistent during testing. The query-understanding and response-generation prompts were also kept separate, allowing each stage to focus on a specific task.

When Semantic Similarity Gets It Right

Several test queries showed the semantic search performing well on its own, without any metadata filtering applied.

"vegan perfume for women" returned strong results, with Commodity Book Personal as the top match. Both the vegan attribute and the women's perfume category were explicitly present in that product's data, which explains why the embedding placed it so close to the query.

"woody earthy scented perfume" returned REPLICA Autumn Vibes as the top result. This product's own description explicitly used the phrase Woody & Earthy Scent, so the match reflected genuinely overlapping language, not a coincidence in the embedding space.

"I want a fresh travel-size perfume for women that would be good for a beach vacation" returned Crescent Beach Perfume Travel Spray as the top match, priced at $29 with a 4.7778 rating. This is a particularly strong example because the query combines several separate ideas at once (fresh, travel-size, women's and beach vacation), and the system connected all of them in one pass. The shared reference to *“beach”* in the query and the product name may also have contributed to its high semantic similarity.

Where Similarity Alone Starts to Struggle

Not every query produced a clean result, and these cases matter for understanding the limits of semantic search on its own.

"I want something cheap that smells expensive" returned a mix of results that included fragrance products but also a deodorant and a candle. This shows that semantic similarity does not enforce an exact product category, and it also does not have a real concept of "cheap" unless price is applied as a hard filter separately.

"I need an elegant perfume for my girlfriend's birthday that isn't too strong" returned results that

were generally relevant, but the phrase *"isn't too strong"* was not something the system could reliably verify, since scent intensity is not represented anywhere in the product data as a field.

"I need a product that won't irritate my sensitive skin" returned semantically relevant results, including REN Evercalm. However, the retrieved product data did not explicitly establish that the product would be non-irritating or suitable for every user with sensitive skin. This was an important finding: semantic relevance does not guarantee suitability or safety, and the system should not be allowed to make these claims unless the underlying data explicitly supports them.

From Retrieval to a Recommendation

Once *final_search_query()* returns a set of candidates, the second Ollama stage, inside *final_response()*, turns them into a recommendation the user actually reads. This stage receives the original user request, the retrieved product information, the metadata, and the distance score for each candidate. The distance score is provided as an internal relevance signal when the model chooses among the retrieved candidates, but neither the score nor other internal retrieval details, such as the embeddings, are shown to the user.

The explanation for each recommended product is built from real product fields: highlights, category, ingredients (when relevant), price, rating, reviews, and size or variation. The final response uses a consistent structure that was specified in the prompt after testing different formats.

This stage also required guardrails to prevent unsupported inferences. Testing surfaced cases where the model made inferences from numbers alone: treating a 5-star rating as evidence of "feminine" or "luxurious," or treating a high review count as evidence of general popularity. The prompt was therefore revised to treat ratings and review counts only as numerical information and not as evidence of characteristics that were not present in the product data. These guardrails reduced unsupported interpretations but did not eliminate them completely. For example, during final testing the model inferred that a fragrance had a "fresh" scent from ingredients such as limonene and linalool, even though the retrieved product information did not explicitly establish that relationship. This reinforces the importance of treating the generated explanation as a separate stage from the underlying retrieval results.

Llama 3.1 (roughly 4.9 GB) was used for both query interpretation and final response generation. . In this project it ran on CPU, so the first call in a session is noticeably slower while the model loads, and subsequent calls are faster. Smaller models were tested as alternatives for the first, query-interpretation Ollama stage. *Qwen 0.6B* was too unreliable for structured numeric extraction, and *Qwen 4B* was better but still invented vague thresholds in some cases. Given this, the practical choice was to keep *Llama 3.1* for generation and avoid overcomplicating the first parser rather than chasing a smaller model that was not yet reliable enough.

Retrieval count was also adjusted during testing. Early testing retrieved 5 candidates from ChromaDB, but the final default was changed to 10 so that the generation stage has more candidates to choose from rather than being limited too early in the pipeline.

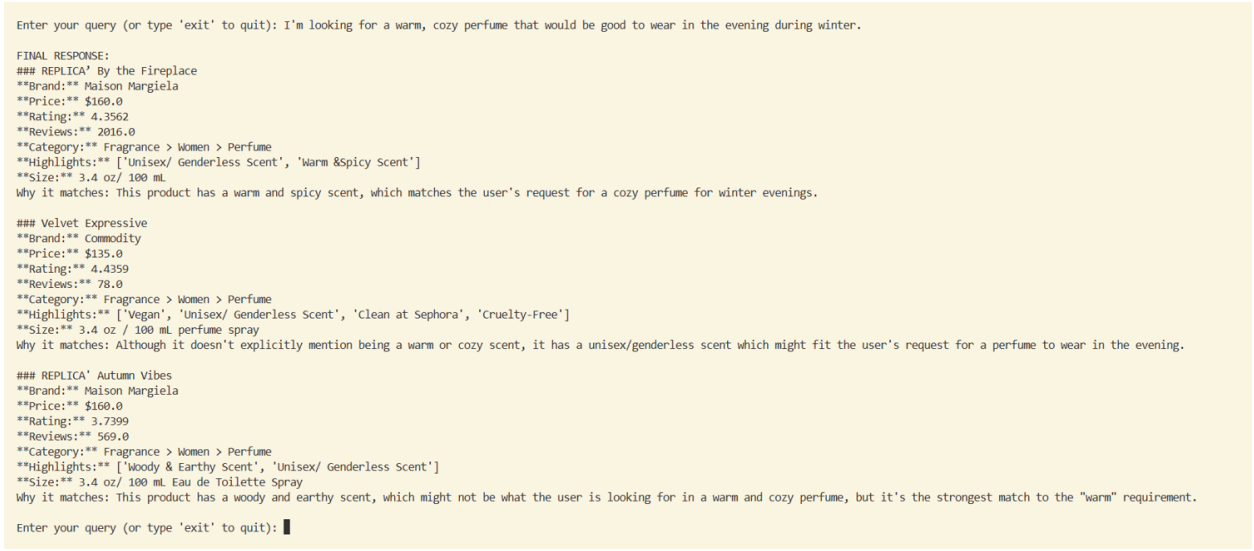


Figure 2. End-to-end prototype output for a natural-language product request.

System Architecture

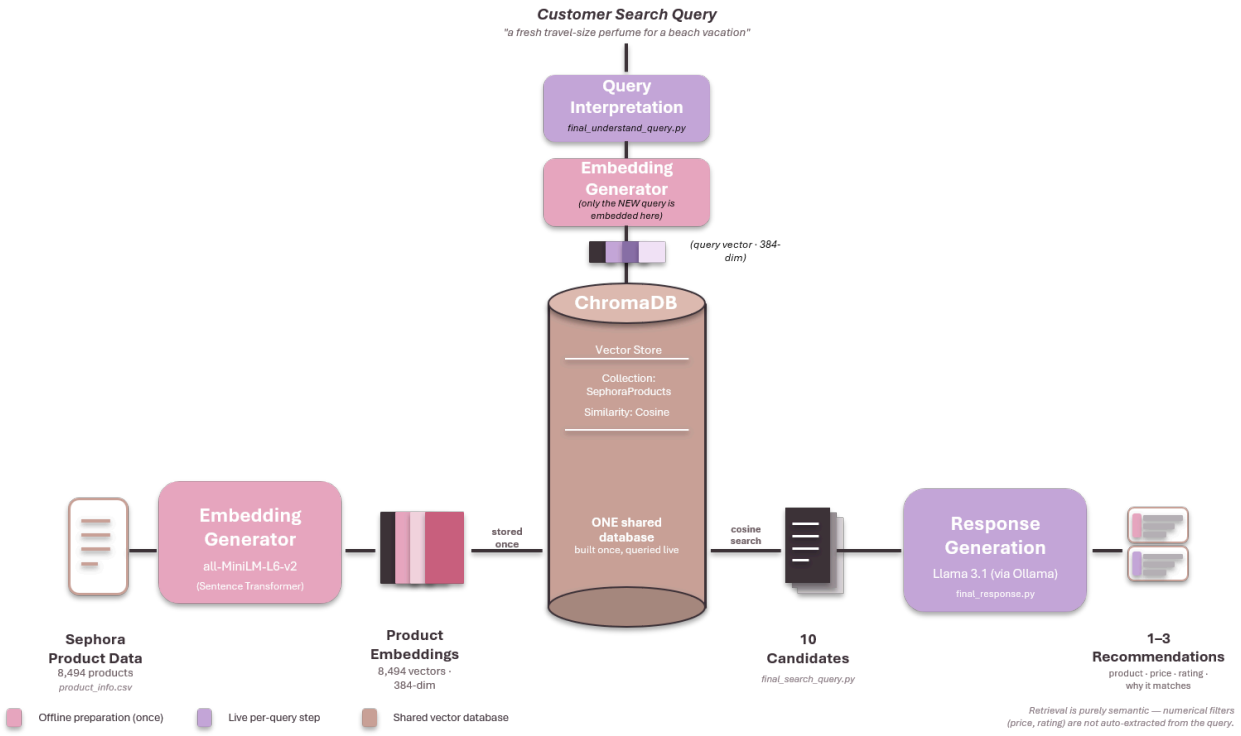


Figure 3. Architecture of the final semantic product search prototype.

The final pipeline is divided into three modules, with each function stored in its own file to keep stages of the system separate: f*inal_understand_query(), final_search_query(), and final_response()*. The main program, FINAL.py, handles user input and contains the *while True* loop that keeps the application running. The structure changed during testing. An earlier version of *user_facing.py*, for example, still contained a hard-coded *while True* testing loop. When the file was imported into the final pipeline, that loop started running immediately and prevented the rest of the application from executing as intended. Separating the testing code from the final function modules resolved this issue and also made each part of the pipeline easier to test and modify independently.

The final process begins when Ollama interprets the user's request and produces the semantic query used for retrieval. Sentence Transformers converts this query into an embedding using the same model used for the stored products, and ChromaDB retrieves the 10 nearest candidates using cosine distance. The retrieved product information and metadata are then passed to Ollama in a second call, which selects and presents the final recommendations. This keeps query interpretation, retrieval, and

response generation as separate stages that can be tested independently.

V. Comparison with Existing Technologies

The development of vector databases does not mean that relational, document, or graph databases have become unsuitable for modern applications. Each is designed around a different way of representing and retrieving data. This becomes clearer when the same product search is considered across technologies. Suppose a customer asks for *"a fresh, elegant perfume for summer under $50."* The price requirement is precise, while "fresh", "elegant", and "for summer" depend on the meaning of the product description. How well a database can answer the request depends on which of these problems it is designed to handle.

The Same Search, Different Database Approaches

A relational database is well suited to the structured parts of this request. If price, rating, brand, category, and inventory are stored as columns, SQL can filter them directly. A condition such as *price_usd <= 50* has a clear meaning and produces a predictable result. Relational databases can also connect products with customers, orders, inventory, and other tables through joins.

The descriptive part of the request is less straightforward. MySQL provides full-text search through *FULLTEXT* indexes and *MATCH()... AGAINST()*, so text search does not have to rely on a basic *LIKE* condition. Traditional full-text search operates on the terms contained in indexed text rather than converting the text into dense semantic embeddings. A retailer could also create fields or tags for attributes such as "fresh", "floral", or "woody". Once those attributes exist, SQL can search them effectively. The difficulty is deciding on the categories beforehand and applying them consistently across the catalogue.

This limitation does not mean that SQL cannot perform similarity calculations. Vector comparisons can be implemented directly in SQL, but whether this is practical depends on the type and scale of the search. Calculating similarity occasionally over a manageable number of vectors is different from repeatedly searching a large collection of high-dimensional embeddings for the nearest matches. Vector databases are designed to index and retrieve these vectors efficiently rather than calculate every

possible comparison during each search.

MongoDB provides another possibility. Its document structure is well suited to product catalogues where records may contain varying attributes. A perfume could contain fragrance notes and concentration, while a skincare product could contain skin type and active ingredients without requiring every product to share exactly the same structure. MongoDB Atlas Vector Search can also index embeddings and combine vector retrieval with filters on other document fields. This means a product document can contain its description, embedding, price, rating, and other information while supporting both semantic and structured retrieval.

For this project, this functionality could have reduced some of the separation between the database and the Python application. Instead of storing the embeddings and selected metadata in a separate ChromaDB collection, they could be stored alongside the other product fields within MongoDB documents. MongoDB also provides interfaces such as Compass for viewing documents, indexes, and queries, which could be more convenient for someone who wants to work directly with the database rather than primarily through Python.

It would not remove the ambiguity found in natural-language queries. If the customer asks for something *"cheap that smells expensive,"* MongoDB can enforce price <= 50 once that rule has been defined, but it does not independently decide that *cheap* means $50. This was the same problem encountered when I initially asked Ollama to translate vague preferences into numerical constraints.

A graph database becomes useful for a separate type of product question. Suppose the retailer wants to know what other products are purchased by customers who buy a particular perfume, which brands those customers prefer, or how products are connected through ingredients and categories. Customers, products, brands, and ingredients could be represented as nodes, with relationships such as *PURCHASED, MADE_BY*, and *CONTAINS* connecting them. A graph database such as Neo4j is designed around these relationships and the paths between them.

The same information can still be represented relationally. Customers, orders, order items, and products can be connected through SQL joins, as demonstrated in the retail database exercises. A graph

database becomes more attractive when those connections are central to the questions being asked and queries repeatedly move through several layers of relationships. For the search “fresh, elegant perfume for summer,” however, there is no obvious relationship to traverse. The main problem is understanding similarity in meaning.

Where Vector Search Adds Something New

That is where the vector approach used in this project becomes useful. Rather than requiring every descriptive preference to exist as a predefined field, the Sentence Transformer represents the query and product descriptions as embeddings. ChromaDB then searches for products located close to the query in that vector space. The embedding model determines how semantic similarity is represented, while ChromaDB provides the storage, indexing, and nearest-neighbor retrieval.

Testing in the previous section demonstrated this difference. Descriptive queries could retrieve relevant products even when several preferences were combined, while vague numerical requirements, category constraints, and safety-related requests exposed the limits of semantic similarity alone.

This is why structured metadata remained important in the ChromaDB implementation. A numerical price constraint should be evaluated independently of semantic similarity. Once a customer explicitly says *“under $50,”* the condition can be handled as a numerical filter. The descriptive portion of the query can continue through semantic retrieval. The two forms of search address different parts of the same request.

Choosing Rather Than Replacing

No single approach is necessary for every type of product query. A query such as *“show me perfumes under $50 with a rating above 4.5”* is already well suited to a relational database. Adding embeddings would introduce complexity without contributing much. A request such as *“find something fresh and elegant for summer”* provides a stronger case for vector search. If the question instead asks how customers, products, brands, and ingredients are connected, a graph structure may be more appropriate. MongoDB offers another option when flexible product documents, metadata, and vector retrieval are better kept within one database environment.

These technologies increasingly overlap in functionality. MySQL HeatWave now supports vector data, embedding generation, and vector distance operations within the MySQL environment. MongoDB has similarly added vector search to its document model. A business that already has either platform may therefore be able to introduce semantic retrieval without adding a standalone vector database.

For this project, ChromaDB was useful as a specialized retrieval layer. Keeping Sentence Transformers, ChromaDB, and Ollama separate required more Python code, but it also allowed each stage to be tested independently. Problems with metadata could be separated from problems with embeddings, while Ollama's interpretation errors could be examined without changing the retrieval layer. A more integrated platform could simplify this architecture for a production application, particularly where an organization already has an established database system.

Database choice ultimately depends on the questions the system needs to answer and the structure of the underlying data. Vector search is most useful when similarity in meaning is central to retrieval, but it does not remove the need for structured information or make existing database technologies obsolete. For an organization considering vector search, the practical question is whether semantic retrieval solves a problem that its current database cannot already handle effectively.

VI. Business Applications and Future Directions

The final prototype demonstrates that semantic search can make product discovery more flexible, but it also leaves several clear directions for further development. The current system works with a relatively small product catalogue, text embeddings, a local language model, and a limited set of metadata. A larger retail application would need to move beyond these boundaries, particularly by incorporating more information about both products and customers and by evaluating the language model more systematically.

Moving Beyond Text

One of the most useful extensions would be image-based product search. The current system represents products through textual information such as their names, categories, highlights, and ingredients. This works when the customer's preference can be expressed in words, but many retail

decisions are visual. A customer may know what a perfume bottle, handbag, shoe, or item of clothing should look like without knowing how to describe its style.

A future version could therefore introduce multimodal embeddings that represent both text and images. Instead of entering only *"a feminine perfume with an elegant bottle,"* a customer could upload an image and search for products that look similar. Text could then refine the visual request, for example *"something like this, but under $50."* This would extend the same similarity-search principle used in the current prototype to another form of product information.

Amazon provides a useful example of how this can work at a much larger scale. Amazon Lens allows customers to upload an image, take a photograph, or scan a barcode to find exact and visually similar products. Its visual search can also be combined with text, allowing someone to upload an image and add a requirement such as changing the colour or material. For beauty and fashion in particular, this creates a form of search that is difficult to reproduce through text alone.

Improving the Language Model

The local use of Llama 3.1 was suitable for the prototype, particularly because Ollama allowed the model to run locally without relying on a paid API. However, inference was relatively slow on the available hardware, and testing identified several cases where the model made unsupported assumptions from the query or retrieved product information. Although prompt changes reduced these problems, they remain an important area for further development.

A future version could compare models systematically using the same set of product queries, checking whether requirements are preserved and whether the resulting recommendations remain supported by the retrieved product information. Unsupported attributes and inaccurate explanations could then be tracked as separate failure cases. A smaller model may still be preferable for the first query-processing stage if it can perform that narrow task reliably, while a stronger model could be reserved for the final recommendation.

Future development could also make the language-model stages more specific to retail queries by testing additional prompt examples or, at a larger scale, fine-tuning an appropriate model on relevant

examples. The goal would not be to let the LLM make every decision. The problems encountered in this project suggest the opposite. Numerical rules and other requirements that can be handled deterministically should remain outside the model where possible. The LLM is most useful for the parts of the interaction that actually require language understanding.

From Product Search to Personal Recommendations

The current system responds to what a customer says in the present query. It does not know what that customer bought previously, which brands they return to, what they have searched for before, or whether they repeatedly choose particular product characteristics. Adding this customer information would move the application from semantic product search toward personalized recommendation.

This is also where some of the database technologies discussed in the previous section could become part of a larger design. A graph database could represent relationships between customers, products, brands, categories, ingredients, searches, and purchases. Semantic retrieval could first identify products that match the meaning of a request, while customer relationships provide another source of evidence for ranking those candidates. Someone searching for *"a light summer fragrance"* might therefore receive results that match the description while also reflecting brands, fragrance families, or products connected to their previous behaviour.

This would not require replacing the vector component. Graph and vector retrieval would answer separate parts of the recommendation problem. The vector search asks which products resemble what the customer is describing. The graph can contribute context about which products are connected to that customer and to similar behaviour. Structured database operations could still enforce price, inventory, location, or other business requirements. A larger system could bring these signals together rather than expecting one database to perform every task.

Amazon's AI Shopping Assistant as a Vision of What This Could Be

Amazon's Rufus provides a useful case study for how far conversational product discovery can develop when it is connected to a much larger retail system. Amazon originally described Rufus as a generative AI shopping assistant trained on its product catalogue and information from across the web,

with the ability to answer product questions, compare options, make recommendations, and support discovery. Amazon has since expanded the assistant considerably. It can incorporate individual shopping activity into recommendations, work with uploaded images, track prices, help with repeat purchases, and support more complex shopping tasks. Rufus was renamed Alexa for Shopping in May 2026.

Amazon provides a useful comparison for understanding what changes when conversational search is connected to a much broader retail environment. My system currently has three main sources of data: the customer's query, the product catalogue, and the metadata stored with each product. Amazon can draw on a much wider context. Its shopping systems use information such as product attributes and customer shopping activity to personalize what customers see, while its shopping assistant can use conversational context and customer activity for recommendations.

The next stage would not simply be a larger vector database. Images would add capabilities that the current text representation cannot capture, while customer behaviour and relationships between products could provide additional evidence for ranking retrieved products. The language model could then help bring these signals into the interaction without becoming responsible for facts that the underlying databases can determine more reliably.

The prototype began with a simple question: how can product search respond to what a customer means rather than only to the words they type? The implementation showed that embeddings and vector search can address part of this problem, particularly for descriptive product requests, but the testing also showed where other information is needed. A more complete retail application would need to combine semantic retrieval with reliable structured constraints and additional content about products and customers. The prototype therefore provides a starting point for a broader recommendation system rather than a complete solution to product discovery.

VII. Bibliography

Amazon. (2025, November 18). *Amazon's next-gen AI assistant for shopping is now even smarter, more capable, and more helpful.* About Amazon. https://www.aboutamazon.com/news/retail/amazon-rufus-ai-assistant-personalized-shopping-features

Chilimbi, T. (2024, October 4). *The technology behind Amazon's GenAI-powered shopping assistant, Rufus.* Amazon Science. https://www.amazon.science/blog/the-technology-behind-amazons-genai-powered-shopping-assistant-rufus

Chroma. (n.d.). *Configure collections.* Chroma Docs. https://docs.trychroma.com/docs/collections/configure

Dascalescu, D., & Hasan, Z. (2023, January 16). *Vector embeddings explained.* Weaviate. https://weaviate.io/blog/vector-embeddings-explained

Galanis, S. (2026, July 3). *Why visual search is reshaping beauty personalization: Pinterest (Opinion).* Retail Insider. https://retail-insider.com/retail-insider/2026/07/the-beauty-discovery-dilemma-why-visual-search-is-reshaping-personalization-pinterest-opinion/

McKinsey & Company. (2026, June 18). *From aisle to algorithm: The beauty categories, channels, and concepts shaping 2030 growth.* https://www.mckinsey.com/industries/consumer-packaged-goods/our-insights/state-of-beauty

Monigatti, L., & Hasan, Z. (2023, August 1). *What is a vector database? Similarity search & semantic search for AI.* Weaviate. https://weaviate.io/blog/what-is-a-vector-database

Omidvar-Tehrani, B., & Guinet, G. (2024, June 13). *Automated evaluation of RAG pipelines with exam*

generation. Amazon Science.

https://www.amazon.science/blog/automated-evaluation-of-rag-pipelines-with-exam-generation

Parton, M. (2025, August 11). *Chatbot cheatsheet: A guide to the AI assistants from Walmart, L'Oréal, Amazon and more.* Modern Retail.

https://www.modernretail.co/operations/chatbot-cheatsheet-a-guide-to-the-ai-assistants-from-walmart-loreal-amazon-and-more/

Pinecone. (n.d.). *A developer's guide to approximate nearest neighbor (ANN) algorithms.*

https://www.pinecone.io/learn/a-developers-guide-to-ann-algorithms/

Statista. (2026). *Gen Z & the beauty market in the United States: Statistics & facts.*

https://www.statista.com/topics/9238/gen-z-and-the-beauty-industry-in-the-united-states/

Statista Research Department. (2025, November 12). *Average yearly spend on beauty products in the United States in 2024, by generation.* Statista.

https://www.statista.com/statistics/1471315/average-yearly-spend-on-beauty-products-us-by-age-group/

Zilliz. (2023, December 15). *Similarity metrics for vector search.* Medium.

https://medium.com/@zilliz_learn/similarity-metrics-for-vector-search-62ccda6cfdd8